

Esercizio 4

Una squadra di muratori deve costruire le pareti di una casa a base quadrata. I muratori sono divisi in due categorie: quelli che mettono i mattoni e quelli che mettono il cemento. Ogni parete viene costruita depositando una striscia di cemento ed una di mattoni, in maniera alternata, e si considera completata quando sono state depositate N file di mattoni.

Ogni muratore effettua ciclicamente le seguenti operazioni: prepara il proprio materiale impiegando 5s per i mattoni e 7s per il cemento, attende il proprio turno in ordine FIFO, inizia a lavorare su una parete per 10s secondi, finisce di lavorare, si riposa per 5 secondi. Quando tutte le pareti saranno completate il lavoro si considera terminato e i muratori vanno via.

Si modelli il sistema descritto in Java, dove i muratori sono dei thread che interagiscono tramite un oggetto *casa* che espone solo i seguenti metodi:

- **boolean inizia(int t):** sospende il muratore fino a quando non è disponibile il lavoro di tipo t , restituisce *false* se non ci sono più lavori da effettuare, *true* altrimenti;
- **void termina(int t):** permette al muratore di comunicare che ha terminato il proprio lavoro.

Si implementino due soluzioni che riproducano il funzionamento del problema sopra descritto utilizzando:

- la classe **Semaphore** del package **java.util.concurrent**
- gli strumenti di mutua esclusione e sincronizzazione del package **java.util.concurrent.locks**

Si scriva infine un **main** d'esempio che, facendo uso di una delle due soluzioni precedenti, inizializzi un oggetto *casa* con $N=20$, 5 muratori che mettono mattoni, 7 muratori che mettono il cemento, e ne avvii l'esecuzione.